

Explicacion detallada del backend Round Robin

Este documento explica el archivo `rr_backend.py`. La idea es entender que hace cada clase, cada funcion, cada variable importante y como se conectan entre si para simular el algoritmo Round Robin.

1. Que es el backend

El backend es la parte del programa que contiene la logica. En este proyecto, el backend no muestra ventanas, botones, tablas ni colores. Eso lo hace el frontend en `rr_frontend.py`.

El backend se encarga de:

- Representar los procesos.
- Representar los bloques del diagrama de Gantt.
- Representar los estados de la cola de listos.
- Simular Round Robin.
- Manejar entradas y salidas.
- Manejar el intercambio o cambio de contexto.
- Calcular los tiempos finales.

En otras palabras: el frontend pregunta "que paso?", y el backend responde con los resultados.

2. Imports del archivo

Al inicio aparece:

```
from __future__ import annotations
```

Esto permite usar anotaciones de tipos de forma mas flexible. Por ejemplo, permite escribir tipos como `List[Process]` sin algunos problemas que pueden aparecer cuando una clase todavia se esta definiendo.

Luego aparece:

```
from collections import deque
```

`deque` significa double-ended queue. Es una cola eficiente. En Round Robin necesitamos una cola de listos: el primer proceso que entra es el primero que sale.

Por eso se usa `deque` en vez de una lista normal. Con `deque`, sacar el primer elemento con `popleft()` es rapido.

Luego aparece:

```
from dataclasses import dataclass, field
```

`dataclass` sirve para crear clases que guardan datos sin escribir mucho codigo repetitivo. Python genera automaticamente el constructor `__init__`, la representacion y otras funciones utiles.

`field` se usa para indicar valores especiales por defecto. En este archivo se usa para crear

listas vacias correctamente.

Finalmente:

```
from typing import Deque, List, Optional
```

Esto ayuda a documentar los tipos de datos:

- List[int]: lista de enteros.
- Deque[Process]: cola de procesos.
- Optional[int]: puede ser un entero o puede ser None.

Estas anotaciones no son obligatorias para que Python funcione, pero hacen el codigo mas claro.

3. Clase Process

La clase Process representa un proceso del sistema operativo.

```
@dataclass
class Process:
```

El decorador @dataclass le dice a Python que esta clase principalmente guarda datos.

3.1 Atributo pid

```
pid: str
```

pid es el identificador del proceso. Por ejemplo:

```
P0
P1
P2
```

Es de tipo str, o sea texto.

3.2 Atributo arrival_time

```
arrival_time: int
```

Es el tiempo de llegada del proceso al sistema. Si P1 llega en 70 ms, entonces:

```
arrival_time = 70
```

El proceso no puede entrar a la cola de listos antes de ese tiempo.

3.3 Atributo cpu_bursts

```
cpu_bursts: List[int]
```

Es una lista con las rafagas de CPU del proceso.

Por ejemplo, si en la tabla se tiene:

```
CPU 1 = 20
CPU 2 = 10
```

```
CPU 3 = 2Q
```

y el quantum es 100 ms, el frontend convierte eso a:

```
cpu_bursts = [200, 100, 200]
```

El backend trabaja con milisegundos, no con Q directamente.

3.4 Atributo `io_bursts`

```
io_bursts: List[int]
```

Es una lista con las entradas/salidas del proceso.

Por ejemplo:

```
E/S 1 = 2Q  
E/S 2 = 2Q
```

con quantum de 100 ms, se convierte a:

```
io_bursts = [200, 200]
```

La relacion normal es:

```
CPU 1 -> E/S 1 -> CPU 2 -> E/S 2 -> CPU 3
```

Por eso normalmente hay una E/S menos que la cantidad de rafagas de CPU.

3.5 Atributo `current_cpu_index`

```
current_cpu_index: int = 0
```

Indica en cual rafaga de CPU va el proceso.

Si vale 0, el proceso esta en CPU 1.

Si vale 1, esta en CPU 2.

Si vale 2, esta en CPU 3.

Se inicia en 0 porque en Python las listas comienzan desde cero.

3.6 Atributo `current_cpu_remaining`

```
current_cpu_remaining: int = 0
```

Indica cuanto le falta al proceso de la rafaga actual de CPU.

Ejemplo:

```
cpu_bursts = [200, 100, 200]
```

Al inicio, `current_cpu_remaining` debe valer 200, porque esa es la primera rafaga pendiente.

Si el proceso ejecuta un quantum de 100 ms, entonces quedaria:

```
current_cpu_remaining = 100
```

3.7 Atributo `completion_time`

```
completion_time: Optional[int] = None
```

Guarda el tiempo en que el proceso termina completamente.

Al principio vale `None`, porque el proceso todavía no ha terminado.

Cuando termina, por ejemplo en 1530 ms, queda:

```
completion_time = 1530
```

3.8 Atributo `first_start_time`

```
first_start_time: Optional[int] = None
```

Guarda el primer momento en el que el proceso entra a CPU.

Esto se usa para calcular la espera inicial:

```
tiempo de espera = primera entrada a CPU - tiempo de llegada
```

Ejemplo:

```
P1 llega en 70 ms
P1 entra por primera vez a CPU en 110 ms
espera = 110 - 70 = 40 ms
```

4. Metodo `__post_init__`

```
def __post_init__(self) -> None:
    self.current_cpu_remaining = self.cpu_bursts[0]
```

Este metodo se ejecuta automaticamente despues de crear un `Process`.

Sirve para inicializar la rafaga actual pendiente.

Si se crea:

```
Process("P0", 0, [200, 100, 200], [200, 200])
```

entonces `__post_init__` coloca:

```
current_cpu_remaining = 200
```

porque `cpu_bursts[0]` es la primera rafaga de CPU.

5. Propiedad `total_cpu_time`

```
@property
def total_cpu_time(self) -> int:
    return sum(self.cpu_bursts)
```

`@property` permite usar el metodo como si fuera un atributo.

En vez de escribir:

```
process.total_cpu_time()
```

se escribe:

```
process.total_cpu_time
```

Esta propiedad suma todas las rafagas de CPU.

Ejemplo:

```
cpu_bursts = [200, 100, 200]
```

Entonces:

```
total_cpu_time = 500
```

6. Propiedad total_io_time

```
@property
def total_io_time(self) -> int:
    return sum(self.io_bursts)
```

Suma todas las entradas/salidas del proceso.

Ejemplo:

```
io_bursts = [200, 200]
```

Entonces:

```
total_io_time = 400
```

Este valor se usa en la formula de clase:

```
tiempo de vuelta = finalizacion - llegada - E/S total
```

7. Propiedad finished

```
@property
def finished(self) -> bool:
    return self.current_cpu_index >= len(self.cpu_bursts)
```

Esta propiedad indica si el proceso ya termino todas sus rafagas de CPU.

Si current_cpu_index es mayor o igual que la cantidad de rafagas, ya no queda CPU pendiente.

Por ejemplo:

```
cpu_bursts = [200, 100, 200]
```

Tiene 3 rafagas. Sus indices son:

```
0, 1, 2
```

Cuando current_cpu_index llega a 3, ya no existe otra rafaga. Por eso el proceso termino.

Actualmente esta propiedad no es indispensable en la simulacion, pero ayuda a expresar la

idea de "proceso terminado".

8. Clase GanttEntry

```
@dataclass
class GanttEntry:
    label: str
    start: int
    end: int
```

Representa un bloque del diagrama de Gantt.

Ejemplo:

```
GanttEntry(label="P0", start=0, end=100)
```

Significa:

Desde 0 ms hasta 100 ms se ejecuto P0.

Tambien puede representar:

```
GanttEntry(label="CS", start=100, end=110)
```

Eso significa cambio de contexto o intercambio.

Tambien puede representar:

```
GanttEntry(label="IDLE", start=500, end=600)
```

Eso significa que la CPU estuvo inactiva.

9. Clase QueueSnapshot

```
@dataclass
class QueueSnapshot:
    time: int
    reason: str
    queue: List[str] = field(default_factory=list)
```

Representa una fotografia de la cola de listos en un momento.

Guarda:

- time: en que tiempo se tomo la fotografia.
- reason: por que ocurrio el cambio.
- queue: que procesos estaban listos.

Ejemplo:

```
QueueSnapshot(
    time=110,
    reason="Se ejecuta P1",
    queue=["P0"]
)
```

Eso significa:

En 110 ms se ejecuta P1 y en cola queda P0.

field(default_factory=list) crea una lista vacia nueva para cada objeto. Esto es importante porque evita que varios objetos compartan la misma lista por accidente.

10. Funcion add_gantt_entry

```
def add_gantt_entry(gantt: List[GanttEntry], label: str, start: int, end: int) ->
None:
```

Esta funcion agrega un bloque al diagrama de Gantt.

Recibe:

- gantt: la lista donde se guardan los bloques.
- label: nombre del bloque, por ejemplo P0, CS o IDLE.
- start: tiempo inicial.
- end: tiempo final.

Primero revisa:

```
if start == end:
    return
```

Si el inicio y el fin son iguales, el bloque dura cero. No tiene sentido dibujarlo.

Luego revisa:

```
if gantt and gantt[-1].label == label and gantt[-1].end == start:
```

Esto pregunta:

- Ya hay bloques en el Gantt?
- El ultimo bloque tiene la misma etiqueta?
- El ultimo bloque termina justo donde empieza este nuevo?

Si todo eso es cierto, no crea otro bloque. Solo alarga el anterior:

```
gantt[-1].end = end
```

Esto evita bloques repetidos innecesarios.

Si no se puede unir, agrega uno nuevo:

```
gantt.append(GanttEntry(label=label, start=start, end=end))
```

11. Funcion queue_labels

```
def queue_labels(queue: Deque[Process]) -> List[str]:
    return [process.pid for process in queue]
```

La cola real guarda objetos Process.

Pero para mostrarla en pantalla no necesitamos todo el objeto. Solo necesitamos el nombre:

P0, P1, P2

Esta funcion convierte:

```
deque([Process("P0"), Process("P1")])
```

en:

```
["P0", "P1"]
```

La parte:

```
[process.pid for process in queue]
```

se llama list comprehension. Es una forma compacta de crear una lista.

12. Funcion enqueue_new_arrivals

```
def enqueue_new_arrivals(  
    processes: List[Process],  
    next_arrival_index: int,  
    current_time: int,  
    ready_queue: Deque[Process],  
) -> int:
```

Esta funcion mete a la cola de listos los procesos que ya llegaron.

Parametros:

- processes: lista de procesos ordenados por llegada.
- next_arrival_index: indice del siguiente proceso que todavia no ha entrado.
- current_time: tiempo actual de la simulacion.
- ready_queue: cola de listos.

La funcion usa:

```
while next_arrival_index < len(processes) and  
processes[next_arrival_index].arrival_time <= current_time:
```

Esto significa:

Mientras exista un proceso pendiente de llegada y su tiempo de llegada sea menor o igual al tiempo actual, entra a la cola.

Luego:

```
ready_queue.append(processes[next_arrival_index])
```

mete ese proceso al final de la cola.

Despues:

```
next_arrival_index += 1
```

avanza al siguiente proceso pendiente.

Finalmente retorna:

```
return next_arrival_index
```

Es necesario devolverlo porque la simulacion necesita recordar cual es el siguiente proceso

que aun no ha llegado.

13. Funcion move_unblocked_processes

```
def move_unblocked_processes(
    blocked: List[tuple[int, int, Process]],
    current_time: int,
    ready_queue: Deque[Process],
) -> None:
```

Esta funcion mueve procesos desde la lista de bloqueados hacia la cola de listos.

Un proceso bloqueado es un proceso que esta haciendo E/S.

La lista blocked guarda tuplas de esta forma:

```
(tiempo_de_desbloqueo, orden, proceso)
```

Ejemplo:

```
(520, 3, P0)
```

significa:

```
P0 termina su E/S en 520 ms.
```

La funcion primero busca los procesos cuya E/S ya termino:

```
unblocked = [item for item in blocked if item[0] <= current_time]
```

Luego deja en blocked solo los que todavia siguen bloqueados:

```
blocked[:] = [item for item in blocked if item[0] > current_time]
```

El uso de blocked[:] = ... modifica la misma lista existente, no crea una lista aparte desconectada.

Despues ordena los desbloqueados:

```
for _, _, process in sorted(unblocked, key=lambda item: (item[0], item[1])):
```

Se ordena por:

- tiempo de desbloqueo.
- orden de insercion.

Eso evita resultados raros cuando dos procesos terminan E/S al mismo tiempo.

Finalmente mete cada proceso desbloqueado a la cola de listos:

```
ready_queue.append(process)
```

14. Funcion next_event_time

```
def next_event_time(
    processes: List[Process],
    next_arrival_index: int,
    blocked: List[tuple[int, int, Process]],
```

```
) -> Optional[int]:
```

Esta funcion se usa cuando la CPU no tiene nada para ejecutar.

Si la cola de listos esta vacia, hay que saber cuando ocurre el proximo evento.

Los eventos posibles son:

- llega un proceso nuevo.
- termina una E/S.

La lista candidates guarda posibles tiempos:

```
candidates: List[int] = []
```

Si todavia queda un proceso por llegar:

```
if next_arrival_index < len(processes):  
    candidates.append(processes[next_arrival_index].arrival_time)
```

agrega su tiempo de llegada.

Si hay procesos bloqueados:

```
if blocked:  
    candidates.append(min(item[0] for item in blocked))
```

agrega el menor tiempo de desbloqueo.

Finalmente:

```
return min(candidates) if candidates else None
```

Si hay eventos candidatos, devuelve el mas cercano. Si no hay ninguno, devuelve None.

15. Funcion simulate_round_robin

Esta es la funcion principal del backend.

```
def simulate_round_robin(  
    processes: List[Process],  
    quantum_size: int,  
    context_switch_size: int,  
    ) -> tuple[List[GanttEntry], List[QueueSnapshot]]:
```

Recibe:

- processes: procesos a simular.
- quantum_size: tamaño del quantum en milisegundos.
- context_switch_size: tamaño del intercambio en milisegundos.

Devuelve:

- lista de bloques para el Gantt.
- lista de estados de la cola de listos.

15.1 Ordenar procesos

```
processes = sorted(processes, key=lambda item: (item.arrival_time, item.pid))
```

Ordena los procesos por tiempo de llegada. Si dos llegan al mismo tiempo, ordena por nombre.
Esto ayuda a que la simulacion sea predecible.

15.2 Crear estructuras principales

```
ready_queue: Deque[Process] = deque()
blocked: List[tuple[int, int, Process]] = []
gantt: List[GanttEntry] = []
snapshots: List[QueueSnapshot] = []
```

ready_queue es la cola de procesos listos.

blocked guarda procesos en E/S.

gantt guarda el diagrama de Gantt.

snapshots guarda la historia de la cola.

15.3 Variables de control

```
time = 0
next_arrival_index = 0
completed_processes = 0
block_sequence = 0
```

time es el reloj de la simulacion.

next_arrival_index indica cual es el siguiente proceso que falta por llegar.

completed_processes cuenta cuantos procesos han terminado.

block_sequence ayuda a ordenar procesos que terminan E/S al mismo tiempo.

15.4 Primer ingreso de procesos

```
next_arrival_index = enqueue_new_arrivals(processes, next_arrival_index, time,
ready_queue)
```

En el tiempo 0, mete a la cola los procesos que llegan en 0.

Luego guarda una foto inicial:

```
snapshots.append(QueueSnapshot(time=time, reason="Estado inicial",
queue=queue_labels(ready_queue)))
```

15.5 Ciclo principal

```
while completed_processes < len(processes):
```

El ciclo se repite hasta que todos los procesos terminen.

Dentro de este ciclo ocurre toda la simulacion.

15.6 Revisar E/S y nuevas llegadas

Al inicio de cada vuelta:

```
move_unblocked_processes(blocked, time, ready_queue)
next_arrival_index = enqueue_new_arrivals(processes, next_arrival_index, time,
ready_queue)
```

Primero se mueven procesos que terminaron E/S.

Luego se agregan procesos nuevos que ya llegaron.

15.7 CPU inactiva

Si no hay procesos listos:

```
if not ready_queue:
```

entonces la CPU no puede ejecutar nada.

Se busca el siguiente evento:

```
event_time = next_event_time(processes, next_arrival_index, blocked)
```

Si no hay evento:

```
if event_time is None:
    break
```

se termina la simulacion.

Si si hay evento, se agrega un bloque IDLE:

```
add_gantt_entry(gantt, "IDLE", time, event_time)
```

Luego el reloj salta hasta ese evento:

```
time = event_time
```

Y se registra en la cola:

```
snapshots.append(QueueSnapshot(time=time, reason="CPU inactiva",
queue=queue_labels(ready_queue)))
```

15.8 Tomar el siguiente proceso

Si hay procesos listos:

```
current = ready_queue.popleft()
```

Saca el primer proceso de la cola.

Esto es Round Robin: el proceso que mas tiempo lleva esperando en la cola se ejecuta primero.

15.9 Registrar primera entrada a CPU

```
if current.first_start_time is None:
    current.first_start_time = time
```

Si el proceso nunca habia entrado a CPU, se guarda este tiempo.

Esto se usa para calcular:

```
tiempo de espera = primera entrada a CPU - llegada
```

15.10 Guardar estado de cola

```
snapshots.append(QueueSnapshot(time=time, reason=f"Se ejecuta {current.pid}",  
queue=queue_labels(ready_queue)))
```

Se guarda que proceso entro a CPU y que quedo en la cola.

15.11 Calcular cuanto ejecuta

```
run_time = min(quantum_size, current.current_cpu_remaining)
```

El proceso ejecuta como maximo un quantum.

Si le falta menos que un quantum, ejecuta solo lo que le falta.

Ejemplo:

```
quantum = 100  
faltante = 60  
run_time = 60
```

15.12 Agregar bloque al Gantt

```
add_gantt_entry(gantt, current.pid, time, time + run_time)
```

Se registra que el proceso se ejecuto desde time hasta time + run_time.

15.13 Avanzar el reloj

```
time += run_time
```

Si antes era 110 y ejecuto 100, ahora el tiempo es 210.

15.14 Restar CPU pendiente

```
current.current_cpu_remaining -= run_time
```

Si al proceso le faltaban 300 ms y ejecuto 100 ms, ahora le faltan 200 ms.

15.15 Variable should_requeue

```
should_requeue = False
```

Esta variable indica si el proceso debe volver a la cola de listos despues de ejecutar.

Debe volver si:

- no termino su rafaga actual de CPU.

No debe volver si:

- termino y pasa a E/S.
- termino todo el proceso.

15.16 Cuando termina una rafaga de CPU

```
if current.current_cpu_remaining == 0:
```

Si ya no le queda CPU en la rafaga actual, hay dos opciones:

- el proceso termino por completo.
- el proceso debe ir a E/S.

Primero:

```
io_index = current.current_cpu_index
current.current_cpu_index += 1
```

io_index indica cual E/S corresponde a la rafaga que acaba de terminar.

Luego se avanza a la siguiente rafaga de CPU.

15.17 Cuando el proceso finaliza

```
if current.current_cpu_index >= len(current.cpu_bursts):
```

Si el indice de CPU ya paso el final de la lista, no quedan mas rafagas.

Entonces:

```
current.completion_time = time
completed_processes += 1
reason = f"{current.pid} finaliza"
```

Se guarda el tiempo de finalizacion y se aumenta el contador de procesos completados.

15.18 Cuando el proceso va a E/S

Si todavia quedan rafagas de CPU:

```
io_duration = current.io_bursts[io_index]
current.current_cpu_remaining = current.cpu_bursts[current.current_cpu_index]
blocked.append((time + io_duration, block_sequence, current))
block_sequence += 1
reason = f"{current.pid} realiza E/S"
```

Se obtiene cuanto dura la E/S.

Se prepara la siguiente rafaga de CPU.

Se mete el proceso en la lista de bloqueados.

El tiempo de desbloqueo es:

```
time + io_duration
```

15.19 Cuando no termina la rafaga de CPU

```

else:
    reason = f"{current.pid} vuelve a listos"
    should_requeue = True

```

Si el proceso no termino su rafaga, significa que se le acabo el quantum.

Entonces debe volver al final de la cola de listos.

15.20 Orden correcto despues de ejecutar

Despues de ejecutar:

```

move_unblocked_processes(blocked, time, ready_queue)
next_arrival_index = enqueue_new_arrivals(processes, next_arrival_index, time,
ready_queue)
if should_requeue:
    ready_queue.append(current)

```

Primero entran procesos que terminaron E/S.

Luego entran procesos que llegaron durante la ejecucion.

Finalmente, si el proceso actual no termino, vuelve al final de la cola.

Este orden es importante para que Round Robin respete los procesos que llegaron mientras otro estaba usando CPU.

15.21 Guardar estado despues del evento

```

snapshots.append(QueueSnapshot(time=time, reason=reason,
queue=queue_labels(ready_queue)))

```

Se guarda una fotografia despues de ejecutar.

15.22 Agregar intercambio

```

if completed_processes < len(processes):
    pending_work = bool(ready_queue or blocked or next_arrival_index <
len(processes))
    if context_switch_size > 0 and pending_work:

```

Si aun quedan procesos por resolver y el intercambio es mayor que cero, se agrega un bloque CS.

```

add_gantt_entry(gantt, "CS", time, time + context_switch_size)
time += context_switch_size

```

Esto representa el tiempo que se pierde cambiando de un proceso a otro.

Despues del intercambio se vuelven a revisar E/S y llegadas:

```

move_unblocked_processes(blocked, time, ready_queue)
next_arrival_index = enqueue_new_arrivals(processes, next_arrival_index, time,
ready_queue)

```

Y se guarda otro estado de cola:

```

snapshots.append(

```

```
        QueueSnapshot(time=time, reason="Fin de intercambio",
                        queue=queue_labels(ready_queue))
    )
```

15.23 Retorno de la simulacion

Al final:

```
    return gantt, snapshots
```

El backend entrega:

- todo el diagrama de Gantt.
- toda la historia de la cola de listos.

16. Funcion calculate_metrics

```
def calculate_metrics(processes: List[Process]) -> List[dict[str, int | str]]:
```

Esta funcion calcula los resultados finales de cada proceso.

Devuelve una lista de diccionarios.

Cada diccionario tiene datos como:

- proceso.
- llegada.
- CPU total.
- E/S total.
- finalizacion.
- tiempo de vuelta.
- tiempo de espera.

16.1 Crear lista de resultados

```
    metrics: List[dict[str, int | str]] = []
```

Aqui se guardan los resultados.

16.2 Recorrer procesos

```
    for process in processes:
```

Calcula las metricas proceso por proceso.

16.3 Ignorar procesos no terminados

```
        if process.completion_time is None:
            continue
```

Si un proceso no tiene tiempo de finalizacion, no se calculan metricas para el.

continue salta al siguiente proceso.

16.4 Retorno estandar

```
standard_turnaround = process.completion_time - process.arrival_time
```

Esta es la formula clasica:

```
retorno estandar = finalizacion - llegada
```

El programa la conserva internamente por si se necesita comparar.

16.5 Espera total estandar

```
total_waiting = standard_turnaround - process.total_cpu_time - process.total_io_time
```

Esta es la espera total clasica.

Resta al retorno estandar el tiempo real de CPU y E/S.

El programa la conserva internamente, aunque la interfaz muestra el criterio usado en clase.

16.6 Espera inicial

```
response = 0 if process.first_start_time is None else process.first_start_time -  
process.arrival_time
```

Calcula cuanto espero el proceso desde que llego hasta que entro por primera vez a CPU.

Esta es la formula que coincide con los datos de clase:

```
tiempo de espera = primera entrada a CPU - llegada
```

Ejemplo:

```
P1 llega en 70  
P1 entra por primera vez en 110  
espera = 110 - 70 = 40
```

16.7 Tiempo de vuelta usado en clase

```
turnaround = standard_turnaround - process.total_io_time
```

El laboratorio de clase uso este criterio:

```
tiempo de vuelta = finalizacion - llegada - E/S total
```

Ejemplo con P0:

```
finalizacion = 1530  
llegada = 0  
E/S total = 400  
tiempo de vuelta = 1530 - 0 - 400 = 1130
```

Ejemplo con P1:

```
finalizacion = 1640  
llegada = 70  
E/S total = 200
```

```
tiempo de vuelta = 1640 - 70 - 200 = 1370
```

16.8 Tiempo de espera usado en clase

```
waiting = response
```

El tiempo de espera mostrado se toma como la espera inicial.

Por eso:

```
P0 = 0  
P1 = 40
```

16.9 Guardar resultados

```
metrics.append(  
    {  
        "pid": process.pid,  
        "arrival": process.arrival_time,  
        "cpu": process.total_cpu_time,  
        "io": process.total_io_time,  
        "completion": process.completion_time,  
        "turnaround": turnaround,  
        "waiting": waiting,  
        "response": response,  
        "standard_turnaround": standard_turnaround,  
        "total_waiting": total_waiting,  
    }  
)
```

Este diccionario guarda todas las metricas.

La interfaz usa principalmente:

- pid
- arrival
- cpu
- io
- completion
- turnaround
- waiting

Tambien quedan guardados:

- response
- standard_turnaround
- total_waiting

Estos ultimos sirven para comparar contra las formulas estandar.

16.10 Retornar metricas

```
return metrics
```

Devuelve todos los resultados al frontend.

17. Ejemplo completo con datos de clase

Datos:

```
Quantum = 100 ms
Intercambio = 10 ms

P0:
llegada = 0
CPU = 2Q, 1Q, 2Q
E/S = 2Q, 2Q

P1:
llegada = 70
CPU = 3Q, 3Q
E/S = 2Q

P2:
llegada = 260
CPU = 6Q
E/S = 0
```

Conversion a milisegundos:

```
P0 CPU = 200, 100, 200
P0 E/S = 200, 200

P1 CPU = 300, 300
P1 E/S = 200

P2 CPU = 600
```

Resultados principales:

```
P0:
finalizacion = 1530
E/S total = 400
tiempo de vuelta = 1530 - 0 - 400 = 1130
tiempo de espera = 0 - 0 = 0

P1:
finalizacion = 1640
E/S total = 200
tiempo de vuelta = 1640 - 70 - 200 = 1370
tiempo de espera = 110 - 70 = 40

P2:
finalizacion = 1860
E/S total = 0
tiempo de vuelta = 1860 - 260 - 0 = 1600
tiempo de espera = 440 - 260 = 180
```

18. Resumen mental de la simulacion

Puedes imaginar el simulador asi:

1. Tengo una cola de procesos listos.
2. Tomo el primero.
3. Lo ejecuto como maximo un quantum.
4. Si termino CPU y le falta E/S, lo bloqueo.
5. Si termino todo, guardo su finalizacion.
6. Si no termino CPU, vuelve al final de la cola.
7. Agrego el intercambio.

8. Repito hasta que todos terminen.

Ese es el corazon del Round Robin.

19. Por que el backend esta separado del frontend

Separar backend y frontend hace que el proyecto sea mas facil de entender.

rr_backend.py responde preguntas como:

```
Que proceso se ejecuta?  
Cuando termina?  
Como queda la cola?  
Cuanto dura el tiempo de vuelta?  
Cuanto dura el tiempo de espera?
```

rr_frontend.py responde preguntas como:

```
Como muestro la tabla?  
Como dibujo el Gantt?  
Como leo los datos escritos por el usuario?  
Como muestro errores?
```

Esto es una buena practica porque evita mezclar logica con interfaz.

20. Que deberias poder explicar en sustentacion

Para sustentar, lo mas importante es poder decir:

```
El backend representa cada proceso con sus rafagas de CPU y E/S.  
La cola de listos se maneja con deque.  
El simulador toma el primer proceso de la cola y lo ejecuta maximo durante un  
quantum.  
Si el proceso no termina la rafaga, vuelve al final de la cola.  
Si termina una rafaga y tiene E/S, pasa a bloqueados.  
Si termina todas sus rafagas, se guarda su tiempo de finalizacion.  
El Gantt se construye agregando bloques de procesos, intercambio e inactividad.  
Las metricas se calculan al final segun las formulas usadas en clase.
```

Con eso puedes explicar tanto la parte tecnica como la parte teorica del laboratorio.